

Benchmarking Service — Developer Guide

A task-oriented guide to driving the Benchmarking Service over its RESTful API. Every `curl` below is derived from the flows we exercised end-to-end (gsm8k single-turn, tau2 multi-turn) on the `ykt3` and `kind-rossoc1` clusters.

- Reference the machine-readable contract in `openapi.json / openapi.yaml`.
 - Design rationale (what the Service can and cannot enact) lives in `SERVICE_DESIGN_DECISIONS.md` and `KUBECTL_DEPENDENCY_INVENTORY.md`.
-

Contents

1. Mental model (read this first).....	2
Endpoint map.....	3
2. Getting a caller token.....	4
3. Onboarding a benchmark (one-time, per cluster/instance).....	6
3.1 Instance config file (instances/<encoded-iss-host>.json).....	6
3.2 Infrastructure resources (baked into the benchmark definitions).....	6
3.3 Workload secrets (provisioned out-of-band as cluster Secrets).....	7
3.4 MLflow + OTEL collector (optional, for reports).....	7
4. Instance-specific Service config (/config).....	8
5. Benchmark lifecycle.....	9
5.0 Discover what's available.....	9
5.1 Deploy (create the MCP tool + A2A agent).....	10
5.2 Wait until Ready.....	11
5.3 Submit a run.....	12
5.4 Poll run status / list runs.....	13
5.5 Get results (report).....	14
5.6 Download output files from S3.....	15
5.7 Tear down.....	15
6. End-to-end examples (validated flows).....	16
7. Known limits & error codes.....	17
8. Extending the catalog (adding or changing a benchmark).....	18
8.1 What a definition holds.....	18
8.2 Add a new benchmark.....	18
8.3 Change an existing benchmark.....	19
8.4 What stays runtime-mutable (for contrast).....	19

1. Mental model (read this first)

The Service is **pure-Python and HTTP-only**. It never calls a cluster API (no kubectl, no kubeconfig). It talks to the Rossoctl/kagenti backend over HTTPS and to Keycloak for tokens. Two consequences shape everything below:

1. **Two independent tokens.**
 - The **caller JWT** you present in `Authorization: Bearer ...` is used *only* for attribution (`preferred_username`) and routing (`iss` selects which instance config applies). It is **never forwarded** upstream.
 - For each request the Service mints its **own** ROPC token (the instance's `benchmarker` credential, password grant) and presents *that* to Rossoctl.
2. **The Service enacts only what HTTP allows.** It deploys workloads (agents/tools), runs benchmarks, reads reports, and exports to S3. It **cannot** create cluster Secrets or overlay per-agent ConfigMaps — those are provisioned out-of-band, and the Service reports on them (see §3 onboarding and the 424 / 422 responses).

Endpoint map

Group	Endpoints	Auth
Identity	GET /hello, GET /healthz (public, unschematized)	caller JWT
Config	GET /config, PUT /config	benchmarker only
Discovery	GET /namespaces, GET /benchmarks, GET /benchmarks/{name}	caller JWT
Raw workloads	GET/POST /agents, GET/DELETE /agents/{ns}/{name}, same for /tools	caller JWT
Benchmark deploy	POST /benchmarks/{name}/deploy, DELETE /benchmarks/{name}/deploy, GET /benchmarks/{name}/status	caller JWT
Run lifecycle	POST /benchmarks/{name}/runs, GET .../runs, GET .../runs/{run_id}	caller JWT
Reporting	GET .../runs/{run_id}/report, GET /benchmarks/{name}/report	caller JWT

2. Getting a caller token

The caller JWT is a normal Keycloak token from the instance's realm. Obtain it via the password grant (the same Direct-Access-Grants flow the dev/test users use). The realm and token endpoint are derived from the instance iss (<iss-origin>/realms/<realm>/protocol/openid-connect/token).

```
# --- Environment (pick the block for your target cluster) ---

# Option A – ykt3 (OpenShift; realm/client "kagenti"), the cluster the
# e2e runs were validated on:
export SVC="https://benchmarking.apps.ykt3.example.com"      #
Benchmarking Service base URL
export KC="https://keycloak.apps.ykt3.example.com"           # Keycloak
base (iss origin)
export REALM="kagenti"                                     # realm
from the iss
export KC_CLIENT="kagenti"                                  # public
client with Direct Access Grants

# Option B – kind-rossoctl (local; realm/client "rossoctl"):
# The instance's iss is
http://keycloak.localtest.me:8080/realms/rossoctl, but in-cluster the
# Service reaches Keycloak via keycloak_backchannel_url. You still
# fetch YOUR token from the
# externally reachable route below (localtest.me resolves to
127.0.0.1).
# export SVC="http://benchmarking.localtest.me:8080"
# export KC="http://keycloak.localtest.me:8080"
# export REALM="rossoctl"
# export KC_CLIENT="rossoctl"

# Attribution matters: config endpoints require preferred_username ==
"benchmarker".
# Any valid realm user works for deploy/run/report.
# Never hardcode the password – supply it out-of-band (env, secret
manager, or an
# interactive prompt). e.g.: read -rs -p "benchmarker password: "
KC_PASS; export KC_PASS
export KC_USER="benchmarker"
export KC_PASS="${KC_PASS:?set KC_PASS in your environment; do not
commit it}"

export TOKEN=$(curl -s
"$KC/realms/$REALM/protocol/openid-connect/token" \
  -d grant_type=password \
  -d client_id="$KC_CLIENT" \
  -d username="$KC_USER" \
  -d password="$KC_PASS" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin))')
```

```
[ "access_token" ] ) )
```

Sanity check: the Service echoes the validated claims + which instance you routed to.

```
curl -s "$SVC/hello" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

GET /hello returns {iss, preferred_username, rossoctl_base_url, claims}. If you get 401, the token is missing/invalid; 403 issuer not in scope means no instance file matches the token's iss (see §3.1).

Why the kind block differs: in-cluster the public iss host is unreachable, so the instance file sets keycloak_backchannel_url and the Service dials *that* for JWKS/ROPC. This only affects the Service's own dialing — your caller token still comes from the externally reachable Keycloak route (Option B above).

3. Onboarding a benchmark (one-time, per cluster/instance)

These steps are **out-of-band** — the Service verifies them but cannot perform them.

3.1 Instance config file (instances/<encoded-iss-host>.json)

One file per instance keys the whole thing to the `iss`. Loaded at startup from `settings.instances_dir`. Shape:

```
{
  "iss": "https://keycloak.apps.ykt3.example.com/realms/kagenti",
  "keycloak_backchannel_url": null,
  "rossoctl_base_url": "https://kagenti-backend.kagenti-
system.svc.cluster.local:8000",
  "service_credential": {
    "client_id": "kagenti",
    "client_secret": "",
    "username": "benchmarker",
    "password": "<benchmarker-password>"
  },
  "mlflow": { "tracking_url": null },
  "s3": { "bucket": null },
  "mcp_endpoint_template": null,
  "agent_endpoint_template": null,
  "workload_otel": null
}
```

- `service_credential` is the ROPC identity the Service presents to Rossoctl (must exist in the realm with `firstName/lastName` set, or Keycloak 400s with “Account is not fully set up”).
- `mcp_endpoint_template` / `agent_endpoint_template` are only needed when the workloads live on a **different** cluster reachable via external routes (use `{service}/{namespace}` placeholders). Leave `null` for co-located in-cluster workloads.
- `mlflow` / `s3` can be seeded here or set later via `PUT /config (§4)`.

3.2 Infrastructure resources (baked into the benchmark definitions)

The Service sends CPU/memory requests+limits in the create body, so no post-create patch is needed. Current values (from `benchmarks/registry.py`):

Workload	requests	limits
MCP tool	500m CPU / 512Mi	4 CPU / 4Gi
A2A agent	500m CPU / 512Mi	4 CPU / 2Gi

3.3 Workload secrets (provisioned out-of-band as cluster Secrets)

The Service has **no Secrets API**. It references these by name in the deploy body; if a Secret is missing the workload never becomes Ready, and the run precheck returns **424** naming exactly what to provision.

Benchmark	Secret (name → key)	Consumed by	Purpose
gsm8k	hf-secret → hf-token	tool	HuggingFace dataset access
gsm8k / tau2 / appworld	openai-secret → apikey	agent (+ tau2/appworld tool)	LiteLLM API key

Provision them on the workload cluster before deploying, e.g.:

```
kubectl -n team1 create secret generic hf-secret --from-literal=hf-token="$HF_TOKEN"
kubectl -n team1 create secret generic openai-secret --from-literal=apikey="$LITELLM_KEY"
```

3.4 MLflow + OTEL collector (optional, for reports)

Reporting is fail-soft: if MLflow client-creds aren't configured, runs still succeed and export to S3, but `report.ndjson` is empty and the report endpoints return **409**. To enable, provision an OTEL collector (forwards agent spans to MLflow) out-of-band and set the MLflow read creds via `PUT /config`.

4. Instance-specific Service config (/config)

GET/PUT /config are **benchmarker-only** (preferred_username == "benchmarker"; else 403). They set only what the Service itself enacts — MLflow (read side) and S3. Attempting to set a workload credential is rejected with **422** (extra="forbid"). Secrets are redacted in responses.

```
# View effective config for your instance (file defaults + any
overrides), secrets redacted.
curl -s "$SVC/config" -H "Authorization: Bearer $TOKEN" | python3 -m
json.tool

# Point the Service's result sink at an S3 bucket + wire up MLflow
read creds.
curl -s -X PUT "$SVC/config" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d '{
    "s3": {
      "bucket": "rossoctl-benchmarking",
      "region": "us-east-1",
      "access_key_id": "AKIA...REDACTED",
      "secret_access_key": "REDACTED",
      "public_read": false
    },
    "mlflow": {
      "tracking_url": "https://mlflow.apps.ykt3.example.com",
      "client_id": "mlflow-client",
      "client_secret": "REDACTED",
      "token_url":
"https://keycloak.apps.ykt3.example.com/realms/kagenti/protocol/openid
-connect/token",
      "insecure_tls": false
    }
  }'
```

Rejected example (workload cred → 422):

```
curl -s -o /dev/null -w '%{http_code}\n' -X PUT "$SVC/config" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d '{"hf-secret": "..."}' # -> 422
```

5. Benchmark lifecycle

The happy path is: **deploy** → **wait for Ready** → **run** → **poll** → **report** → **download from S3**.

5.0 Discover what's available

```
curl -s "$SVC/benchmarks" -H "Authorization: Bearer $TOKEN" | python3  
-m json.tool  
curl -s "$SVC/benchmarks/gsm8k" -H "Authorization: Bearer $TOKEN" |  
python3 -m json.tool  
curl -s "$SVC/namespaces" -H "Authorization: Bearer $TOKEN" | python3  
-m json.tool
```

Three benchmarks are registered: gsm8k (single-turn), tau2 (multi-turn, runs a server-side user-simulator LLM), appworld (registered; see §7).

5.1 Deploy (create the MCP tool + A2A agent)

POST /benchmarks/{name}/deploy creates both the shared MCP tool (exgentic-mcp-<name>) and the agent (exgentic-a2a-<agent>-<name>[-<experiment>]). Returns 201.

```
curl -s -X POST "$SVC/benchmarks/gsm8k/deploy" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d '{
    "agent": "tool_calling",
    "namespace": "team1",
    "experiment": "default"
  }' | python3 -m json.tool
```

Request fields (DeployBenchmarkRequest):

Field	Default	Meaning
agent	tool_calling	agent flavor (only tool_calling today)
model	null	deploy-time model, baked into agent env; null → benchmark default (openai/Qwen3.6-35B-A3B)
namespace	team1	target namespace
experiment	default	non-default suffixes the agent name so variants coexist
authbridge_enabled	false	inject the AuthBridge sidecar with the cluster-default pipeline (layer-2 knob)
plugin_preset / plugins / plugin_config_file	null	rejected with 422 — layer-3 pipeline composition needs a kubectl ConfigMap overlay the Service can't do

Model swap (run #4 pattern): the deploy endpoint always (re)creates the *shared* tool, so re-deploying with a new model 409s on the existing tool. Deploy the model-swapped agent alone via POST /agents under a distinct experiment instead:

```
# Generate the agent body with build_agent_request semantics, then
POST /agents directly.
curl -s -X POST "$SVC/agents" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d '{
    "name": "exgentic-a2a-tool-calling-gsm8k-azmini",
    "namespace": "team1",
```

```

        "container_image": "ghcr.io/exgentic/exgentic-a2a-
tool_calling",
        "image_tag": "latest",
        "env_vars": [
            {"name": "MCP_URL", "value": "http://exgentic-mcp-gsm8k-
mcp.team1.svc.cluster.local:8000/mcp"},
            {"name": "LLM_MODEL", "value": "openai/Azure/gpt-4o-mini"},
            {"name": "EXGENTIC_SET_AGENT_MODEL", "value":
"openai/Azure/gpt-4o-mini"},
            {"name": "OPENAI_API_KEY", "value_from": {"secret_key_ref":
{"name": "openai-secret", "key": "apikey"}}}
        ],
        "service_ports": [{"name": "http", "port": 8080,
"target_port": 8000}]
    }' | python3 -m json.tool

```

5.2 Wait until Ready

```

curl -s "$SVC/benchmarks/gsm8k/status?
namespace=team1&agent=tool_calling&experiment=default" \
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool

```

Returns `tool_ready` / `agent_ready` booleans plus raw `readyStatus`. Poll until both are true. If a workload stays not-Ready, the run precheck (§5.3) will name the missing Secret.

5.3 Submit a run

POST /benchmarks/{name}/runs runs a **cluster-API-free precheck** (tool+agent deployed and Ready) then returns **202** with a run_id. Prechecks:

- 409 — not deployed (POST .../deploy first).
- 424 — deployed but not Ready; the message names required Secret(s) (e.g. hf-secret).

```
export RUN=$(curl -s -X POST "$SVC/benchmarks/gsm8k/runs" \
-H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
-d '{
  "agent": "tool_calling",
  "namespace": "team1",
  "experiment": "default",
  "max_tasks": 10,
  "max_parallel_sessions": 1,
  "timeout_seconds": 600
}' | python3 -c 'import sys,json; print(json.load(sys.stdin)
["run_id"])' )
echo "run_id=$RUN"
```

Run fields (RunRequest) are all **run-time** knobs:

Field	Default	Notes
max_tasks	1	number of benchmark tasks to evaluate
max_parallel_sessions	1	concurrency
timeout_seconds	300	whole-run wall-clock ceiling; raise for large tau2 runs
agent / namespace / experiment	—	must match a deployed agent
model	null	not forwarded per-session; model is fixed at deploy time

Timeout tuning (learned e2e): tau2 with max_tasks=20

max_parallel_sessions=4 exceeded the 900s default and failed; re-running with timeout_seconds=1800 succeeded (~915s).

5.4 Poll run status / list runs

One run's full state (status, summary, per-task results, artifacts once exported).

```
curl -s "$SVC/benchmarks/gsm8k/runs/$RUN" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

All runs for this benchmark scoped to your instance.

```
curl -s "$SVC/benchmarks/gsm8k/runs" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

status moves pending → running → succeeded|failed. On success, summary carries {total, succeeded, evaluated_pass, pass_rate, wall_seconds}. A simple poll loop:

```
while ;; do
  S=$(curl -s "$SVC/benchmarks/gsm8k/runs/$RUN" -H "Authorization: Bearer $TOKEN" \
    | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d["status"])' )
  echo "status=$S"; [ "$S" = succeeded ] || [ "$S" = failed ] &&
  break; sleep 15
done
```

5.5 Get results (report)

Two report views, both reading structured records from MLflow (return 409 if MLflow isn't configured for the instance — see §3.4):

Per-run report: records filtered to this run's session ids, plus its S3 artifacts.

```
curl -s "$SVC/benchmarks/gsm8k/runs/$RUN/report" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Experiment rollup: time-windowed aggregates across an experiment's runs.

```
curl -s "$SVC/benchmarks/gsm8k/report?experiment=default&window_h=3" \
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Per-run report is {run_id, benchmark, experiment, trace_count, records[], artifacts[]}; each record (MLflowTraceRecord) has per-session timing breakdown, LLM/tool latencies + token counts, infra CPU/mem, and the evaluation outcome.

5.6 Download output files from S3

When the instance has an S3 bucket set, each completed run is exported (fail-soft) and its objects appear in the run state under `artifacts[]` (and in the per-run report). Layout:

```
<prefix>/<preferred_username>/<encoded-iss>/<benchmark>/<run_id>/
  run.json          # the run summary (always written)
  report.ndjson     # one record per line (empty if MLflow
unconfigured)
  report.parquet    # analytics-friendly (only when there are
records)
```

Grab the keys/URLs straight from the run state, then download:

```
# Artifact keys + URLs for the run.
curl -s "$SVC/benchmarks/gsm8k/runs/$RUN" -H "Authorization: Bearer
$TOKEN" \
  | python3 -c 'import sys,json; [print(a["format"], a["key"],
a["url"]) for a in json.load(sys.stdin)["artifacts"]]'

# Download the whole run prefix (private bucket → use AWS creds;
public_read → plain curl the url).
PREFIX=$(curl -s "$SVC/benchmarks/gsm8k/runs/$RUN" -H "Authorization:
Bearer $TOKEN" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)
["artifacts_prefix"])' )
aws s3 cp "s3://rossoctl-benchmarking/$PREFIX/" ./results/ --recursive
```

5.7 Tear down

```
curl -s -X DELETE "$SVC/benchmarks/gsm8k/deploy?
namespace=team1&agent=tool_calling&experiment=default" \
  -H "Authorization: Bearer $TOKEN" -o /dev/null -w '%{http_code}\n'
# -> 204
```

Deletes the agent and the shared tool. For a model-swap variant deployed via `POST /agents`, delete it with `DELETE /agents/{namespace}/{name}` (the shared tool is left for other experiments).

6. End-to-end examples (validated flows)

These map to the canonical parameterized runs and were validated on ykt3 with S3 export.

```
# Run #2 – gsm8k, 10 tasks (validated: 9/10 pass)
curl -s -X POST "$SVC/benchmarks/gsm8k/runs" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 10, "max_parallel_sessions": 1, "timeout_seconds": 600}'
```

```
# Run #3 – gsm8k, 50 tasks, 4-way concurrency (validated: 42/50, ~190s)
curl -s -X POST "$SVC/benchmarks/gsm8k/runs" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 50, "max_parallel_sessions": 4, "timeout_seconds": 900}'
```

```
# Run #9 – tau2, 10 tasks (multi-turn; deploy tau2 first)
curl -s -X POST "$SVC/benchmarks/tau2/deploy" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"namespace": "team1"}'
curl -s -X POST "$SVC/benchmarks/tau2/runs" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 10, "max_parallel_sessions": 1, "timeout_seconds": 1200}'
```

```
# Run #10 – tau2, 20 tasks, 4-way concurrency (validated: 17/20, ~915s – needs raised timeout)
curl -s -X POST "$SVC/benchmarks/tau2/runs" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 20, "max_parallel_sessions": 4, "timeout_seconds": 1800}'
```

7. Known limits & error codes

Situation	Code	What to do
Missing/invalid bearer	401	fetch a fresh token (§2)
iss not in any instance file	403	add/repair the instance config (§3.1)
/config as non-benchmarkers	403	authenticate as the benchmarkers user
Setting a workload cred via /config	422	provision it as a cluster Secret instead (§3.3)
plugin_preset/ plugins/ plugin_config_file on deploy	422	not enactable HTTP-only (needs kubectl ConfigMap overlay); use authbridge_enabled=true for the sidecar with the cluster-default pipeline
Run before deploy	409	POST .../deploy first
Run while not Ready	424	provision the named Secret(s), wait for Ready (§5.2)
Report with MLflow unconfigured	409	set MLflow read creds via PUT /config (§3.4)
Upstream Rossoctl/MLflow failure	502	transient upstream issue; retry

AuthBridge plugin presets (runs 4–8 of the ibac comparison): layer-3 plugin composition and on_error selection require overlaying the per-agent authbridge-config-`<agent>` ConfigMap via kubectl — which the HTTP-only Service cannot do — so those are honestly rejected with 422. The one enactable knob is `authbridge_enabled=true` (injects the sidecar with the cluster-default pipeline).

appworld: registers and deploys correctly via the Service, but its MCP image hangs on its own per-task Venv-service health timeout (120s), independent of the Service. Treat appworld as externally blocked until that image is fixed.

8. Extending the catalog (adding or changing a benchmark)

The benchmark catalog is **code, not runtime data** — a static BENCHMARKS dict in `src/benchmarking_service/benchmarks/registry.py`. There is no database, config file, or admin API behind it, and the HTTP surface over the catalog is **read-only** (GET `/benchmarks`, GET `/benchmarks/{name}`). Adding or changing a benchmark is therefore a **source change + tests + image rebuild + redeploy**, deliberately: the definition pins container images, dataset env, and resource limits that must move in lockstep with the workload images, so every change is a reviewable, git-tracked, image-versioned artifact rather than mutable state that could drift per instance.

See the co-located contributor note `src/benchmarking_service/benchmarks/README.md` for the field-by-field reference and the per-benchmark env gotchas.

8.1 What a definition holds

Each entry is a `BenchmarkDefinition`: `name`, `mcp_image` (+ `mcp_image_tag/mcp_port/mcp_path`), `tool_env`, `tool_resources`, `default_model`, `user_simulator` (multi-turn flag), and an `agents` map of `BenchmarkAgentSpec` (per-agent `container_image` + `extra_env` + `resources`). Secret references are declared with the `_secret_env(env, secret, key)` helper; `required_secrets()` derives the actionable 424 precheck message from them automatically.

8.2 Add a new benchmark

1. **Add a `BenchmarkDefinition` entry** to BENCHMARKS in `registry.py`:
 - `mcp_image` (+ `tag/port/path`) — the MCP tool image.
 - `tool_env` — BENCHMARK_NAME, the LiteLLM base URL, and any secret refs via `_secret_env`.
 - `agents={...}` — one `BenchmarkAgentSpec` per flavor. The `tool_calling` A2A image is shared across all current benchmarks; usually you reuse it and only the name gets a `-<benchmark>` suffix.
 - `user_simulator=True` **only** for multi-turn benchmarks — this is what makes `build_tool_request` inject `EXGENTIC_SET_BENCHMARK_USER_SIMULATOR_MODEL` into the MCP pod so the server-side simulator shares the run's model.
 - Any benchmark-quirk env — mind the documented gotchas: `gsm8k` needs `EXGENTIC_SET_BENCHMARK_RUNNER=direct`; `tau2` needs `EXGENTIC_SET_BENCHMARK_ACTION_TIMEOUT=1000`; `appworld` **rejects** that same `action-timeout` override and crashes at startup if it's present.
2. **Add tests** in `tests/test_benchmarks.py` — assert `build_tool_request` / `build_agent_request` emit the expected images, env, secrets, and resources, mirroring the existing patterns.
3. **Rebuild + bump the image tag**, redeploy the Service. The new benchmark then appears in GET `/benchmarks` and is deployable/runnable at `/benchmarks/<newname>/...` with **no client change** — the request bodies are benchmark-agnostic (see §5).

8.3 Change an existing benchmark

Same mechanism — edit the entry (bump `mcp_image_tag`, change `default_model`, add an env var, adjust `tool_resources`), update tests, rebuild, redeploy. Because `required_secrets()` is derived from `tool_env` + the chosen agent's `extra_env`, changing a secret reference automatically updates the 424 “this benchmark requires secret(s): ...” precheck message — no separate wiring.

8.4 What stays runtime-mutable (for contrast)

Per-instance state is *not* in the catalog and does not require a rebuild: instance config (files under `settings.instances_dir`, keyed by `iss`) and the benchmarker-only `PUT /config` overrides (MLflow read creds + S3). The benchmark catalog is intentionally the immutable, version-pinned part.